

svTidy et le « formula-masking » : une alternative intéressante à {dplyr} et {tidyr} pour remanier vos jeux de données

Philippe Grosjean ¹

Guyliann Engels ¹²

Résumé (max 300 mots)

Le package {svTidy} représente une alternative à {dplyr} et {tidyr} du Tidyverse pour remanier des tableaux de données. Il expose une interface similaire avec des fonctions qui ont des noms et des arguments similaires à celles des deux autres packages. Cependant, il remplace la « tidy evaluation », l'approche non standard d'évaluation des arguments dans le Tidyverse, par soit une évaluation standard, soit le « formula-masking ». Le « formula-masking » consiste à utiliser des objets formula de R pour spécifier des instructions non standard de manière similaire à ce que fait le Tidyverse. Cette approche a de nombreux avantages qui seront développés dans cette présentation : code plus rapide, directement compatible avec le R CMD check pour les packages R, instructions plus explicites et pas de conflits entre le nom des variables et le nom des arguments dans les fonctions.

Mots-clefs (3 à 5) : Manipulation de données - SciViews - svTidy - formula-masking - Tidyverse

Développement

Le Tidyverse ne doit plus être présenté : il est largement utilisé en remaniement de données sous R. Son succès tient à sa modularité et à sa syntaxe facile à lire et à écrire. Cependant, la « tidy evaluation » qui est à la base de l'implémentation des fonctions des packages {dplyr} et {tidyr} entre autres, n'a pas que des qualités. Elle est référentiellement non-transparente et impose d'adopter des solutions contournées et complexes pour permettre le remplacement de variables dans le code Tidyverse. C'est le « data-masking » et son mécanisme d'injection de variables à l'aide, entre autres, des opérateurs !! et { }. De plus, la « tidy evaluation » n'est pas compatible avec les outils de vérification du code dans les packages R (R CMD check) et la solution proposée –précéder les variables du data frame par .data\$, et les variables d'environnement par .env\$– rend au final le code Tidyverse plus compliqué et répétitif que ne le ferait une simple évaluation standard des arguments de la fonction.

Nous proposons une approche alternative utilisant la même syntaxe que les fonctions {dplyr} et {tidyr}, mais avec, au choix, une évaluation standard des arguments (et donc, référentiellement

¹Service d'écologie numérique, Institut Complexys & Infortech, Université de Mons, Belgique, philippe.grosjean@umons.ac.be

²Service d'écologie numérique, Institut Complexys & Infortech, Université de Mons, Belgique, guyliann.engels@umons.ac.be

transparente), ou avec une interface formule (appelée ici « formula-masking »). Cette dernière est intéressante à plus d'un titre :

1. Il s'agit d'objets standard du langage R, bien connus des utilisateurs dans d'autres contextes, par exemple avec les fonctions `lm()` et `glm()`.
2. Les formules dans R sont invariablement construites à l'aide de l'opérateur `~` qui ne sert à rien d'autre. Il est ainsi très facile de déterminer lorsqu'une formule est utilisée dans du code R, et donc, lorsque le « formula-masking » entre en jeu. Cela enlève l'ambiguïté d'une évaluation standard ou non standard du code.
3. Le remplacement de variables est facile et rapide dans les formules à l'aide de la primitive `substitute()`. Cela permet d'écrire un code plus concis et plus rapide que la « tidy evaluation » accompagnée du « data-masking » pour la substitution de variables dans le Tidyverse.
4. Grâce aux formules, il est possible de contourner le problème de la création de nom de variables qui seraient identiques au nom d'arguments de la fonction. Dans le Tidyverse, la « solution » utilisée consiste à précéder les arguments qui peuvent entrer en conflit par un point : `.by=`, par exemple dans `filter()` ou `slice()`, alors qu'ailleurs c'est `by=` qui est utilisé, comme dans `slice_head()` ou `slice_min()`. Mais du coup, on ne peut plus appeler une variable `.by`, et cela ne fait que déplacer le problème. Ce dernier se rencontre aussi pour `.data=`, `.groups`, etc. créant par là-même une hétérogénéité mal venue dans l'API Tidyverse. Avec le « formula-masking », il est possible de nommer les variables à créer avec des fonctions telles que `select()` ou `mutate()` sans entrer en conflit avec le nom des arguments de la fonction.

Le package `{svTidy}` réimplémente complètement la quasi-totalité des fonctions `{dplyr}` et `{tidyr}`, en acceptant les deux options (évaluation standard ou formula-masking). Ses fonctions sont également souvent plus rapides que leurs équivalents Tidyverse, surtout pour les petits jeux de données, grâce à l'infrastructure plus légère du « formula-masking ». `{svTidy}` représente au final une alternative à `{dplyr}` et `{tidyr}` qui permet d'écrire un code plus explicite, et donc, moins susceptible d'erreurs cachées. Aussi, le code en mode interactif est identique ou quasi-identique à celui à l'intérieur d'une fonction et totalement compatible avec le R CMD check lorsqu'il est inclus dans un package R. Il est ainsi plus facile de passer d'un script à une fonction, puis à un package pour distribuer ses œuvres.